H21 Platform — System Architecture

— Infrastructure topology: Frontend · Backend · Auth · Database · Storage · External APIs

BROWSER / CLIENT

React App (Next.js)
- Server + Client components
- App Router (Next.js 16)
- TailwindCSS + Radix UI
- React Query · TipTap editor
- React Hook Form + Yup

Stripe.js Elements
- Credit card input (PCI DSS)
- stripe.confirmPayment()
- Client-side only — no card data

Supabase Realtime Client
- WebSocket channel listener
- messages · notifications

Google Analytics 4
- Client-side events SDK

Wistia Video Player
- Embed player (JS)
- Video stream via Akamai CDN
- embed-ssl.wistia.com

Next.js Image / Link
- remotePatterns allowlist
- supabase.co · wistia.com

next.config.ts
- Image remote patterns
- Server Actions 100MB limit
- Turbopack · pdfjs external

env-cmd
- .env.develop / .env.local
- .env.staging / .env.prod

Security Boundary
No secret keys in browser
STRIPE_PUBLISHABLE (pk_*) only
Anon key for client Supabase
All sensitive ops via server

HTTPS · Next.js

clientSecret

RSC · Server Actions

VERCEL PLATFORM (harvest21.com · staging-m2.harvest21.com)

EDGE NETWORK — CDN + Middleware (runs at Edge runtime before any server code)

Vercel CDN / Static Assets
- Public JS/CSS/font bundles
- Next.js Image Optimization
- CORS on /_next/image route
- Static pages (ISR/SSG)

Edge Middleware (middleware.ts)
- $` Maintenance mode redirect (Vercel Flags SDK)
- $a CORS — OPTIONS preflight (allowlist: harvest21.com, staging, localhost)
- $b Auth session refresh supabase.auth.getClaims() · HTTP-only cookies
- $c Route guards /admin !' role check · unauthenticated !' redirect /

JWT verify / getClaims()

APPLICATION LAYER — Next.js Serverless Functions

Next.js SSR Pages (app/ router)
- 30+ page routes
- / · /missionaries · /[page_url]
- /donate · /settings · /messages
- /admin/* (role-gated)

Server Actions (actions.ts files)
- ~15 actions.ts modules
- Admin CRUD · Settings · Social
- Donations · Messaging · Media
- Run server-side — no HTTP overhead

lib/ Business Services
- donationHelpers · stripeHelpers
- notificationHelpers · pageHelpers
- gmailMailerService · tokenHelpers
- wistiaService · fileUploadHelpers

Rate Limiter (lib/rateLimit.ts)
- In-memory IP counter
- 10 req / 60s on /api/donate/*
- Returns HTTP 429

Auth API /api/auth/*
- POST signin — email+pw · status check · sign out on inactive
- POST signup-supporter · POST signout
- POST activate-account · POST send-activation-email · POST send-reset-email

PostgREST API (anon + service role)

Payments API /api/donate/* /api/stripe-connect/*
- POST create-payment-intent — rate limited, email validated
- Stripe PaymentIntent / Subscription with metadata
- POST create-account (Stripe Connect) · GET account-status

Stripe Webhook /api/webhooks/stripe
- HMAC signature verify (stripe.webhooks.constructEvent)
- payment_intent.succeeded · invoice.paid · customer.subscription.deleted

Media API /api/page-media · /api/storage · /api/photos · /api/vid
- Signed upload URLs (UUID-prefixed filenames)
- fileName sanitization: /[^a-zA-Z0-9.-]/g !' "_"

Storage REST API (signed URL)

Wistia API /api/wistia/*
- token · upload · upload-credentials · folders · move-video
- OAuth: /api/wistia/callback — code exchange !' token storage
- Config · Projects · Delete endpoints

Misc APIs
- contact · user-profile · get-page-id · check-unpublished-owner · missionaries/[id]/followers/*

SUPABASE PLATFORM (kstznftkyihjchkfkcah.supabase.co)

Auth Service (@supabase/ssr)
- JWT sessions — signed Supabase tokens
- HTTP-only cookies (path=/ · sameSite=lax · secure in prod)
- supabase.auth.getClaims() on every request (middleware)
- supabase.auth.getUser() for admin routes
- Custom JWT (jose HS256) for invitations — 72h expiry, type="activation" field
- Password reset via admin.generateLink() + Gmail email

PostgreSQL Database (30+ tables, 8 domains)
- Row Level Security (RLS) enabled on all tables
- Domains: Auth/Identity · Organizations · Donations · Content · Social · Messaging · Admin · Settings
- Key tables: missionaries · churches · agencies · pages · page_donations · donors · follow_requests · prayer_requests · conversations · messages · notifications · page_media · page_widgets
- Service role key bypasses RLS for privileged server ops (webhooks, admin actions)
- Anon key used for client/SSR — RLS policies enforced via auth.uid()

Storage (h21-dev bucket)
- File types: profile photos · banners · PDFs · video thumbnails
- Access: public URLs (images) + signed upload URLs (writes)
- File path: /{entity_type}/{entity_id}/{folder}/{uuid}-{sanitized_name}
- Wistia handles large video — Supabase storage for images/docs only

Realtime (WebSocket pub/sub)
- Channels: messages table, notifications table
- Browser subscribes via supabase.channel() — direct WS to Supabase (NOT via Vercel)
- Live DM delivery · unread badge updates

Key RLS Policies

missionary_followers	INSERT: auth.uid()=user_id · UPDATE: owner or admin role
messages / conversations	SELECT: only conversation participants can read
users	SELECT: any authenticated · UPDATE: admin role only
page_donations	INSERT/SELECT: service role (webhook, server actions)
pages	SELECT: public · INSERT/UPDATE: owner or service role

Supabase Edge Functions (supabase/functions/)
- fetch_missionaries_overview · fetch_missionary_pages_by_region · get_profile
- Called from lib/ server utilities for heavy data queries
- Run in Deno runtime on Supabase infrastructure

Stripe API (server-side)

Webhook (HMAC signed)

WebSocket (Realtime — direct browser ↔ Supabase, bypasses Vercel)

Wistia REST API

EXTERNAL SERVICES (called from Vercel server-side only — never exposed to browser)

Stripe
- Node.js SDK (server-only)
- PaymentIntents (one-time)
- Subscriptions (monthly)
- Connect Express accounts
- Webhooks !' /api/webhooks/stripe
- Refunds · Disputes handling
- Idempotency keys on PI create
- Mode: TEST (pk_test_ / sk_test_)

Gmail / Nodemailer
- gmailMailerService.ts
- Auth: App Password (SMTP)
- Emails: activation · reset
- Donation receipts
- Missionary notifications
- Contact form replies
- From: no-reply@harvest21.com
- Mailgun configured (env) — not yet active

Wistia Video
- OAuth 2.0 token exchange
- /api/wistia/callback route
- Video upload (server proxy)
- Project/folder management
- Delivery: Akamai CDN
- Domains: embed-ssl.wistia.com · fast.wistia.com
- Player: Browser !" Akamai direct

Google Analytics 4
- Client-side SDK
- GoogleAnalytics component
- Page view tracking
- No server-side events
- (analytics.tsx in components/)

Vercel Flags SDK
- @flags-sdk/vercel adapter
- flags.ts: maintenanceMode flag
- Evaluated at Edge runtime
- Blocks all routes when ON
- !' redirects to /maintenance
- FLAGS_SECRET · FLAGS env keys

Connection legend: !" HTTPS · RSC · Server Actions · !" JWT auth / DB queries (PostgREST) · !" Storage API (signed URL) · !ã Stripe/Gmail/Wistia API calls · !á Stripe Webhook (HMAC verified) · !" WebSocket Realtime (direct)

USER ROLE HIERARCHY

SUPER ADMIN (role = 1)
- Full platform access
- Bypass all RLS via service role
- Manage missionaries/orgs/users
- View all transactions
- Admin dashboard + settings

STAFF ADMIN (role = 2)
- Admin dashboard access
- Manage entities & users
- Approve pages · review reports
- View donations + analytics

MISSIONARY
- Own public profile page
- Manage media, widgets, content
- Approve/reject follow requests
- Receive donations · view receipts
- Enable Stripe Connect payout
- Send/receive DMs

CHURCH / AGENCY / COLLEGE REP
- Manage own organization page
- View affiliated missionaries
- Send/receive DMs

SUPPORTER / DONOR
- Follow missionaries (requires approval)
- Send donations (one-time or monthly)
- Send DMs to missionaries
- View own donation history

GUEST (unauthenticated)
- View public missionary profiles
- View public church/agency pages
- Browse missionaries by region
- Make one-time donations (no login required)
- Cannot follow, DM, or view settings

PERMISSION MATRIX

Action / Resource	Guest	Supporter	Missionary	Org Rep	Admin	Super Admin
View public missionary profiles	'	'	'	'	'	'
View public org pages	'	'	'	'	'	'
Browse missionaries by region	'	'	'	'	'	'
One-time donation (no login)	'	'	'	'	'	'
Monthly recurring donation	'	'	'	'	'	'
Follow a missionary (w/ approval)	'	'	'	'	'	'
Send direct messages	'	'	'	'	'	'
View own donation history	'	'	'	'	'	'
Cancel own recurring donation	'	'	'	'	'	'
Create/edit own profile page	'	'	own	own	'	'
Submit page for review	'	'	own	own	'	'
Approve/reject page review	'	'	'	'	'	'
Manage media + widgets	'	'	own	own	'	'
Approve follower requests	'	'	own	'	'	'
Enable Stripe Connect payout	'	'	own	'	'	'
View all donations	'	'	'	'	'	'
Manage all users / missionaries	'	'	'	'	'	'
Admin dashboard access	'	'	'	'	'	'
Bypass RLS (service role)	'	'	'	'	'	'
Homepage + footer settings	'	'	'	'	'	'
Feature flag management	'	'	'	'	'	'

SESSION & TOKEN LIFECYCLE

Supabase Session JWT (@supabase/ssr)
- Issued by: Supabase Auth on signInWithPassword()
- Stored in: HTTP-only cookie (secure, sameSite=lax)
- Refreshed by: supabase.auth.getClaims() on every request
- Claims: user_id, email, role, iat, exp
- Validated by: middleware !' updateSession()

Custom Activation JWT (jose / HS256)
- Issued by: POST /api/send-activation-email
- Algorithm: HS256 signed with JWT_SECRET env var
- Expiry: 72 hours
- Claims: { userId, email, type:"activation" }
- Used once: POST /api/activate-account then discarded

Supabase Service Role Key (server-only)
- Used by: getSupabaseAdmin() — server-side only
- Bypasses RLS — all tables accessible
- Used for: webhooks, admin actions, onboarding ops
- Never sent to browser / never in NEXT_PUBLIC_ vars

Stripe Webhook Signature (HMAC-SHA256)
- Every webhook verified: stripe.webhooks.constructEvent()
- Secret: STRIPE_WEBHOOK_SECRET env var
- Missing/invalid signature !' 400 rejection immediately

Stripe API Keys (server-only)
- STRIPE_SECRET_KEY (sk_*) — server-only; Stripe SDK init
- STRIPE_PUBLISHABLE_KEY (pk_*) — browser; Stripe.js Elements

Feature Flag Auth (Vercel Flags SDK)
- FLAGS_SECRET: HMAC for flag override requests
- FLAGS: server token for Vercel Toolbar
- maintenanceMode flag evaluated at Edge — kills all routes

Admin Authentication Flow (/admin/* routes)

1. Request hits Edge Middleware
2. supabase.auth.getClaims() — checks JWT cookie
3. If no user !' redirect to /
4. If user present !' supabase.auth.getUser() (full user fetch)
5. Query users table: SELECT role WHERE user_id = auth.uid()
6. If role "` 1 or 2 !' redirect to /
7. Passes request to Next.js admin page / server action
8. Admin pages use getSupabaseAdmin() !' service role !' RLS bypassed

Account Status Guard (POST /api/auth/signin)

After Supabase auth !' query users.status field

status = "inactive" !' signOut() immediately !' return 403 + accountDisabled:true

H21 Platform — Environments & Deployment

— Dev · Staging · Production · CI/CD pre-push pipeline · Key inventory · Security posture

ENVIRONMENT CONFIGURATIONS

DEVELOPMENT

npm script: npm run dev (env-cmd -f .env.develop)
URL: http://localhost:3000
Supabase: kstznftkyihjchkfkcah.supabase.co
Stripe: TEST mode (pk_test_ / sk_test_)
Email: Gmail — no-reply@harvest21.com
Flags: FLAGS + FLAGS_SECRET env keys
Env file: .env.develop

- Hot reload (Turbopack)
- RLS still enforced
- Stripe test cards only
- Dev Supabase project

STAGING

npm script: npm run dev:staging (env-cmd -f .env.staging)
URL: https://staging-m2.harvest21.com
Supabase: Separate Supabase project
Stripe: TEST mode (staging Stripe keys)
Email: Gmail or Mailgun (mg.harvest21.com)
Flags: Vercel Flags SDK — staging project
Env file: .env.staging

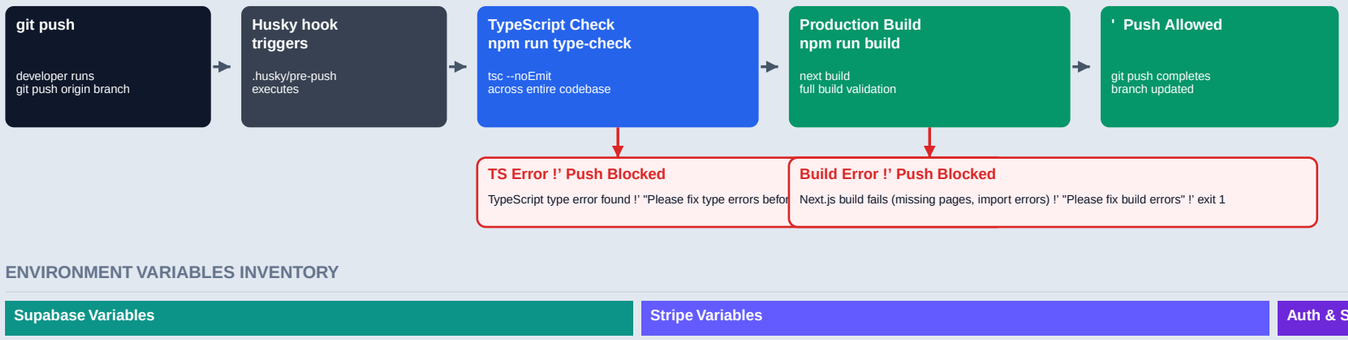
- CORS allowlist includes staging domain
- Pre-push hook runs here
- Full build + type check required
- Test webhooks via Stripe CLI

PRODUCTION

npm script: npm run build:prod (env-cmd -f .env.prod)
URL: https://harvest21.com
Supabase: tuoqghemdpiimyrkjzrf.supabase.co
Stripe: LIVE mode (pk_live_ / sk_live_)
Email: Gmail — no-reply@harvest21.com
Flags: Vercel Flags SDK — production
Env file: .env.prod (not committed)

- HTTPS-only cookies (secure:true)
- Live Stripe payments
- Production Supabase + RLS
- Real webhook secret

CI / CD — PRE-PUSH VALIDATION PIPELINE (.husky/pre-push)



ENVIRONMENT VARIABLES INVENTORY

Supabase Variables

NEXT_PUBLIC_SUPABASE_URL	public	PostgREST + Storage API base URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	public	Client-side RLS-enforced access
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	public	New-style publishable key
SUPABASE_SERVICE_ROLE_KEY	secret	Bypasses RLS — server-only

Stripe Variables

NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	public	Browser — Stripe.js Elements only
STRIPE_SECRET_KEY	secret	Server SDK — create PIs, subs
STRIPE_WEBHOOK_SECRET	secret	Verify incoming webhook signatures

Auth & Security Variables

JWT_SECRET	secret	Sign activation tokens (HS256)
FLAGS_SECRET	secret	Vercel feature flag HMAC
FLAGS	secret	Vercel Flags server token

Email Variables

GMAIL_USER	secret	no-reply@harvest21.com
GMAIL_APP_PASSWORD	secret	Google App Password (SMTP)
MAILGUN_API_KEY	secret	Mailgun API (configured, not active)
MAILGUN_DOMAIN	config	mg.harvest21.com
MAILGUN_EMAIL_NO_REPLY	config	noreply@mg.harvest21.com

SECURITY POSTURE SUMMARY

TRANSPORT

- HTTPS everywhere (Vercel enforced)
- HTTP-only cookies (JS cannot read)
- secure flag on all cookies in prod
- CORS allowlist (not wildcard on API)
- HMAC signature on all Stripe webhooks

AUTHENTICATION

- Supabase Auth — JWT sessions
- Custom JWT for invitations (HS256)
- Account status checked on every login
- Admin role checked on every /admin/* request
- No secrets in NEXT_PUBLIC_ variables (except pub keys)

DATA

- Row Level Security on all tables
- Service role key: server-only usage
- Parameterized queries via PostgREST
- File names sanitized before storage
- HTML escaped before email insertion

KNOWN GAPS

- Rate limiter: in-memory (resets on restart)
- storage/signed-upload: no auth check
- JWT_SECRET fallback to hardcoded value
- users SELECT: open to all authenticated
- No ESLint in pre-push (only tsc + build)